

Phase 2 Report – MPI-Based Distributed Word Count

1. Introduction

This report describes Phase 2 of the **ParaWord** project, which extends a Phase 1 shared-memory word count application (OpenMP) into a **distributed-memory MPI program**. The original application counts the number of words in a large text file and measures performance. In Phase 2, the goal is to transform this design into a distributed parallel implementation using MPI, implement domain decomposition, use non-blocking communication to overlap computation and communication, and evaluate strong and weak scaling as well as network latency and bandwidth.

In the MPI version, the input file is partitioned into contiguous blocks that are processed by multiple MPI ranks. Each rank counts word starts in its local block, while a halo exchange handles word boundaries that are split across ranks. The results are then combined using MPI collectives.

2. Design of Decomposition

2.1 Global Problem Definition

The global problem is to count the number of words in a text file `sample2.txt`. The file is read as a **sequence of bytes** of total length N . A word is defined as a maximal sequence of non-whitespace characters, where whitespace includes space, tab, newline, carriage return, and form feed.

2.2 1D Block Decomposition

We use a 1D block decomposition over the file bytes. Let p be the number of MPI ranks. We compute:

```
base = N / p
rem  = N % p

// rank r
local_N = base + (r < rem ? 1 : 0);
start   = r * base + (r < rem ? r : rem);
```

Ranks $0 \dots \text{rem}-1$ receive $\text{base}+1$ bytes, and the remaining ranks receive base bytes. The `start` index identifies the first byte of the global file owned by rank r . Each rank stores its local block in a separate buffer `local` of size `local_N`.

2.3 Local State per Rank

- **Local buffer:** `local[0 ... local_N-1]` – the file segment assigned to this rank.
- **Halo characters:** `left_halo` and `right_halo` – characters just before and after the local block in the global file.
- **Local word count:** `local_words` – number of word starts detected in this block.

2.4 File Distribution Strategy

To keep the design simple and explicit, **rank 0** is responsible for reading the entire file and distributing chunks via `MPI_Send` to other ranks. Each nonzero rank receives its assigned chunk using `MPI_Recv`. This approach makes communication patterns visible and easy to reason about in the report, and it satisfies the requirement to use point-to-point MPI primitives.

3. Communication Pattern

3.1 Point-to-Point Communication

The program uses blocking point-to-point communication for **data distribution**:

- Rank 0 uses `MPI_Send` to send each rank its local chunk.
- Ranks 1...p-1 use `MPI_Recv` to receive these chunks.

This phase happens once at startup and is followed by the main word-count computation.

3.2 Non-Blocking Halo Exchange

Words may be split across the boundary between two ranks. For example, the last characters of rank `r` and the first characters of rank `r+1` may form a single word. To handle this correctly, each rank performs a **halo exchange** of one character with its neighbors.

The non-blocking pattern is:

1. Post `MPI_Irecv` for `left_halo` (from `rank-1`) and `right_halo` (from `rank+1`), if those neighbors exist.
2. Post `MPI_Isend` of the first and last local characters to left and right neighbors.
3. Compute interior word starts in the local block while communication progresses.
4. Call `MPI_Waitall` to ensure halos are received.
5. Use `left_halo` to decide whether index `0` in the local buffer begins a word.

This pattern overlaps communication with computation and demonstrates the required use of non-blocking MPI primitives (`MPI_Isend`, `MPI_Irecv`, `MPI_Waitall`).

3.3 Collectives

Several MPI collectives are used:

- **MPI_Bcast** – rank 0 broadcasts the total file size N to all ranks so that each can compute its local block.
- **MPI_Reduce** – used twice:
 - To compute the global word count as a sum of all `local_words`.
 - To compute the maximum runtime across ranks (for strong/weak scaling plots).

4. Scaling Plots

4.1 Strong Scaling

In the strong scaling experiment, the total file size is kept fixed while the number of MPI ranks is increased (e.g. 1, 2, 4, 8 ranks). For each configuration, the runtime $T(p)$ is measured as the maximum time across ranks (using **MPI_Reduce**). The speedup and efficiency are computed as:

$$\begin{aligned}\text{Speedup } S(p) &= T(1) / T(p) \\ \text{Efficiency } E(p) &= S(p) / p\end{aligned}$$

The speedup curve shows sublinear behavior: performance improves as the number of ranks increases, but the curve gradually flattens. The efficiency plot decreases as p grows, which is expected due to communication overhead, load imbalance, and sequential portions such as file I/O on rank 0. Amdahl's Law helps explain this: even a small sequential section limits the achievable speedup at higher process counts.

4.2 Weak Scaling

In the weak scaling experiment, the amount of work per rank is kept approximately constant. For example, each rank processes a fixed-size file segment (e.g. 5 MB), and the global file size grows proportionally with

the number of ranks. Ideally, the runtime should remain roughly constant as p increases, because each rank is doing the same amount of work.

In practice, the runtime slightly increases with the number of ranks. This reflects the cost of:

- Additional halo exchanges as more boundaries are introduced.
- Increased cost of the final reduction over more processes.
- Possible contention on the storage system when multiple ranks read or create files.

The weak scaling results illustrate the balance between communication cost and parallelism for larger distributed systems.

5. Latency and Bandwidth Results

A separate MPI program was implemented to measure **network latency and bandwidth** via a ping-pong test between two ranks. For each message size (e.g. 8 bytes up to 1 MB), the program exchanges messages for a fixed number of iterations, measures the total round-trip time, and derives:

```
latency = (round-trip time / iterations) / 2  
bandwidth = message_size_bytes / latency
```

The latency plot shows that for small messages, communication cost is dominated by a nearly constant startup overhead rather than by payload size. As the message size grows, latency increases but not linearly, reflecting the fixed per-message overhead.

The bandwidth plot shows low effective bandwidth for tiny messages; as the message size increases, the bandwidth improves and eventually approaches a plateau determined by the underlying hardware. These results help explain why, in the main word count application, it is preferable to send fewer, larger messages (e.g. one block per rank) rather than many tiny messages.

6. Discussion of Bottlenecks

6.1 File I/O and Rank 0 Serialization

A major bottleneck in the current design is that **rank 0** performs all file I/O: it reads the entire file and sends chunks to other ranks. As the file size increases, this step becomes a sequential bottleneck, limiting speedup according to Amdahl's Law. Furthermore, the `MPI_Send` operations from rank 0 to all other ranks introduce additional serialization at startup.

6.2 Communication Overhead and Halo Exchange

Although the halo exchange uses non-blocking communication and overlaps with interior computation, there is still unavoidable overhead tied to message startup and synchronization. As the number of ranks increases, more boundaries and halo exchanges are introduced, which is reflected in the efficiency drop in strong scaling and the slightly increasing runtime in weak scaling.

6.3 Load Imbalance and Text Irregularity

The decomposition is based on equal-sized byte ranges, not on equal numbers of words. Some ranks may receive parts of the file with more punctuation or whitespace, while others get dense text. This can lead to minor load imbalance where some ranks finish earlier but must still wait at barriers for others, reducing overall efficiency.

6.4 Network Characteristics

The latency and bandwidth benchmark shows that small messages are expensive and that the network reaches peak throughput only for reasonably large messages. While the main application is designed to use a small number of relatively large messages, any additional fine-grained messaging (for example, more complex halo patterns) would be significantly impacted by these network limitations.

7. Reflection: OpenMP vs. MPI

7.1 Programming Effort and Complexity

The Phase 1 OpenMP implementation required only minor modifications to the sequential code: adding `#pragma omp parallel` and `#pragma omp for` directives to parallelize loops. Data sharing is implicit in a shared-memory environment, which simplifies development.

In contrast, the MPI implementation in Phase 2 required explicit design choices for domain decomposition, message passing, non-blocking communication, and collectives. The programmer must carefully reason about data ownership, communication patterns, and synchronization. This increases complexity but provides much more control and portability across distributed-memory systems.

7.2 Performance Characteristics

On a single node with multiple cores, the OpenMP version can be very efficient because it avoids explicit communication and leverages the cache hierarchy of a shared-memory system. For moderately sized text files, OpenMP may outperform MPI due to the lack of message-passing overhead.

However, OpenMP is limited to a single shared-memory machine. MPI, by contrast, can scale across multiple nodes and handle much larger data sets by distributing the file across ranks. Even though MPI introduces communication overhead, the ability to use more processes and more memory makes it the appropriate model for truly large-scale workloads.

7.3 Future Improvements

Several improvements could further enhance the MPI implementation:

- Using **parallel file I/O** (e.g. MPI-IO) so that each rank reads its own segment directly from disk.
- Combining MPI with OpenMP (**hybrid MPI+OpenMP**) to reduce the number of MPI ranks and communication volume while still exploiting multi-core nodes.

Team Members:

Ahmed Sameh 202202151

Hesham Ashraf 202201477

- Improving load balancing by partitioning the file based on logical units (paragraphs or lines) rather than purely byte ranges.

Overall, Phase 2 demonstrates the trade-offs between OpenMP and MPI and provides hands-on experience with designing and evaluating a distributed parallel system.